There are a few later choices that conflict technically with earlier ones—especially **singleton Page Objects vs parallel execution**. For a scalable framework, I'll resolve those conflicts in favor of **thread safety and your explicit simplicity goal**, and I'll clearly explain those decisions rather than silently making assumptions.

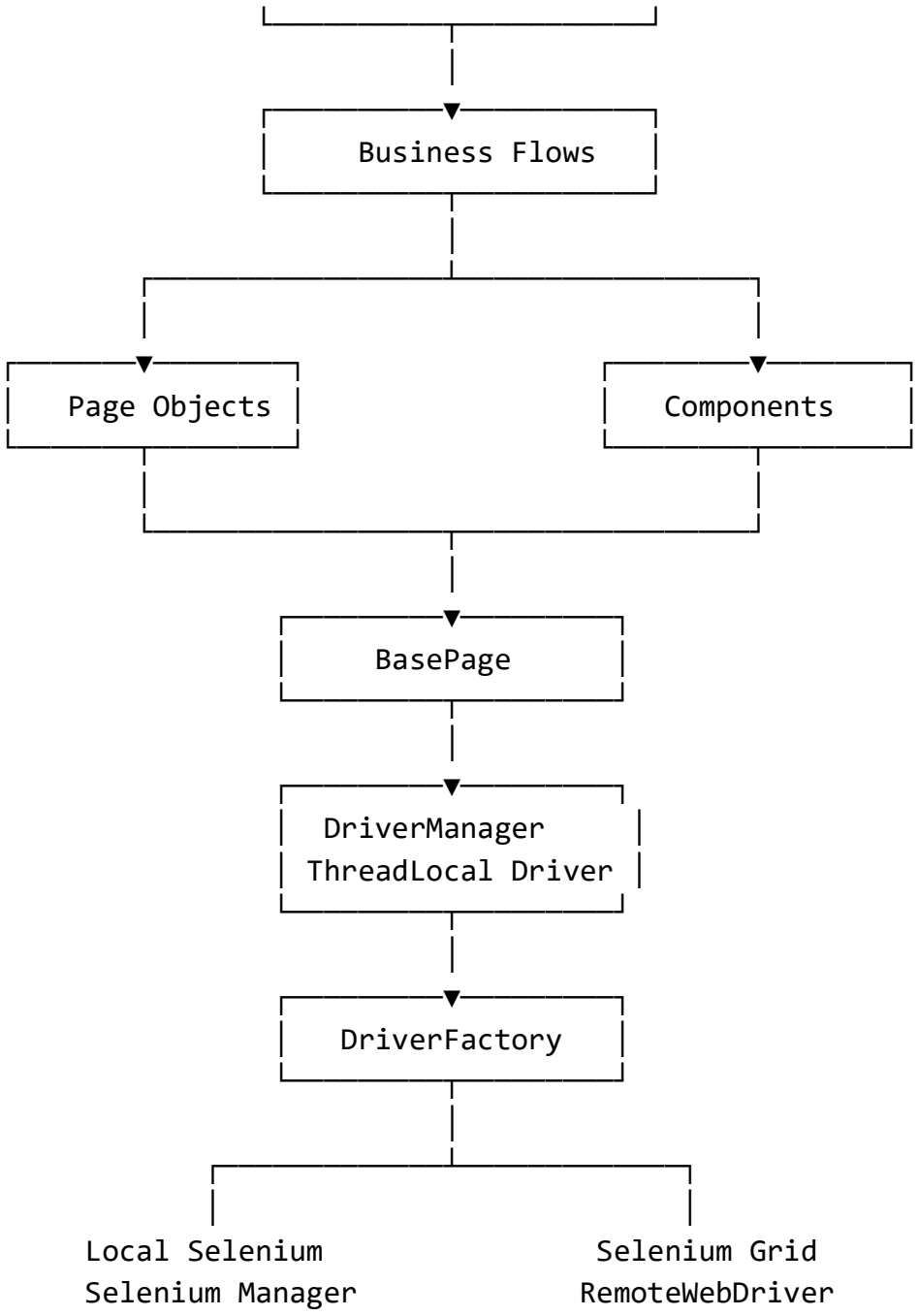
1. Finalized Requirements

Area	Your selection
Language	Java 21
UI	Selenium WebDriver
API	Rest Assured — architected, but core implementation first
Test framework	TestNG
Build	Maven + Surefire
Design	Maintainability + simplicity
POM	Page Object Model + BasePage + reusable Components
Locators	By locators
Waits	Explicit waits + reusable waits
Browser	Chrome + Firefox
Execution	Local + Selenium Grid
Parallelism	TestNG parallel methods
Driver	`ThreadLocal<WebDriver>`
Driver creation	Factory pattern
Local drivers	Selenium Manager
Grid	`RemoteWebDriver`
Data	Excel + JSON + TestNG DataProvider
Excel organization	Module-based sheets
JSON organization	Module-based files
Environment data	Environment sections in Excel/JSON
Data validation	Fail fast
Config	Environment `.properties` files
Config precedence	TestNG XML → environment properties → Maven/system properties
Reporting	Allure
Failure screenshots	Yes, failure only
Logging	Log4j2
Retry	TestNG RetryAnalyzer, 2 retries

Failed rerun	`testng-failed.xml`
CI/CD	CI-independent Maven design
Secrets	AWS Secrets Manager
DB	Architecture-ready, not implemented initially
Authentication	Multiple strategies architecturally
OTP	Splunk UI, same browser/tab strategy
JavaScript	Only as fallback
Alerts	Direct Selenium API
Frames	Not initially required
Windows/tabs	Not initially required
Dropdowns	Application-specific
API schema validation	Not initially required
API response-time threshold	3 seconds
SSL	Strict
Browser certificates	Strict
Browser versions	Configured version, fallback to installed version
Test-data special chars	Preserve
Whitespace	Preserve
Unicode	UTF-8
Static analysis	None
Page validation	Exact URL/title
Page inheritance	BasePage only
Components	Reusable independent Component Objects
Business flows	Module-based Flow classes
API clients	Module-specific clients
API client return	Rest Assured `Response`
Initial implementation priority	**Core framework first**

2. Architecture

The framework will use this high-level flow:



Supporting infrastructure:

```
Configuration
      |
      ├── Environment Properties
      ├── TestNG Parameters
      └── Maven/System Properties
```

```
Test Data
      │
      ├── Excel
      ├── JSON
      └── TestNG DataProvider

Reporting
      │
      ├── Allure
      ├── Failure Screenshots
      └── TestNG Listener

Infrastructure
      │
      ├── Log4j2
      ├── RetryAnalyzer
      ├── WebDriverListener
      └── FrameworkException
```

3. Recommended Project Structure

```
selenium-automation-framework/
│
├── pom.xml
│
├── testng.xml
│
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/
│   │   │       └── automation/
│   │   │           └── framework/
│   │   │
│   │   │           ├── constants/
│   │   │           │   └── FrameworkConstants.java
│   │   │
```

```
│   │   │           ├── config/
│   │   │           │   ├── ConfigManager.java
│   │   │           │   └── ConfigLoader.java
│   │   │
│   │   │           ├── driver/
│   │   │           │   ├── DriverFactory.java
│   │   │           │   ├── DriverManager.java
│   │   │           │   └── DriverType.java
│   │   │
│   │   │           ├── exceptions/
│   │   │           │   └── FrameworkException.java
│   │   │
│   │   │           ├── pages/
│   │   │           │   └── BasePage.java
│   │   │
│   │   │           ├── components/
│   │   │           │   └── BaseComponent.java
│   │   │
│   │   │           ├── waits/
│   │   │           │   └── WaitUtils.java
│   │   │
│   │   │           ├── listeners/
│   │   │           │   ├── TestListener.java
│   │   │           │   └── WebDriverEventListener.java
│   │   │
│   │   │           ├── retry/
│   │   │           │   └── RetryAnalyzer.java
│   │   │
│   │   │           ├── data/
│   │   │           │   ├── ExcelDataReader.java
│   │   │           │   ├── JsonDataReader.java
│   │   │           │   └── DataMapper.java
│   │   │
│   │   │           └── utils/
│   │   │               ├── ScreenshotUtils.java
│   │   │               └── JsonUtils.java
│   │   │
│   │   └── resources/
│   │       └── config/
```

```
│   │           ├── qa.properties
│   │           └── uat.properties
│   │
│   └── test/
│       ├── java/
│       │   └── com/
│       │       └── automation/
│       │           └── tests/
│       │
│       │               ├── base/
│       │               │   └── BaseTest.java
│       │               │
│       │               ├── pages/
│       │               │   └── LoginPage.java
│       │               │
│       │               ├── components/
│       │               │   └── HeaderComponent.java
│       │               │
│       │               ├── flows/
│       │               │   └── login/
│       │               │       └── LoginFlow.java
│       │               │
│       │               ├── data/
│       │               │   └── LoginData.java
│       │               │
│       │               └── ui/
│       │                   └── LoginTest.java
│       │
│       └── resources/
│           ├── testdata/
│           │   ├── excel/
│           │   │   └── login.xlsx
│           │   └── json/
│           │       └── login.json
│           │
│           └── suites/
│               ├── ui-smoke.xml
│               ├── ui-regression.xml
│               └── api-regression.xml
```

```
│
└── target/
```

4. Maven Configuration

Your framework should centralize dependency versions using Maven properties, as you selected.

```
<properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>

    <selenium.version>4.x.x</selenium.version>
    <testng.version>7.x.x</testng.version>
    <restassured.version>5.x.x</restassured.version>
    <allure.version>2.x.x</allure.version>
    <log4j.version>2.x.x</log4j.version>
    <poi.version>5.x.x</poi.version>
    <jackson.version>2.x.x</jackson.version>
    <aws.sdk.version>2.x.x</aws.sdk.version>
</properties>
```

The actual versions should be pinned to mutually compatible stable releases when the project is created rather than using floating versions.

Core dependencies:

```
<dependencies>

    <!-- Selenium -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>${selenium.version}</version>
    </dependency>

    <!-- TestNG -->
    <dependency>
```

```
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>${testng.version}</version>
</dependency>

<!-- Rest Assured -->
<dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>${restassured.version}</version>
</dependency>

<!-- Apache POI -->
<dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>${poi.version}</version>
</dependency>

<!-- Jackson -->
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>${jackson.version}</version>
</dependency>

<!-- Log4j2 -->
<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-api</artifactId>
    <version>${log4j.version}</version>
</dependency>

<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>${log4j.version}</version>
</dependency>
```

```
    <!-- Allure TestNG -->
    <dependency>
        <groupId>io.qameta.allure</groupId>
        <artifactId>allure-testng</artifactId>
        <version>${allure.version}</version>
    </dependency>

</dependencies>
```

Surefire:

```
<build>
    <plugins>

        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.x.x</version>

            <configuration>
                <suiteXmlFiles>
                    <suiteXmlFile>testng.xml</suiteXmlFile>
                </suiteXmlFiles>

                <properties>
                    <property>
                        <name>parallel</name>
                        <value>methods</value>
                    </property>

                    <property>
                        <name>threadCount</name>
                        <value>4</value>
                    </property>
                </properties>
            </configuration>
        </plugin>

    </plugins>
</build>
```

5. Framework Constants

```
package com.automation.framework.constants;

public final class FrameworkConstants {

    private FrameworkConstants() {
        // Prevent object creation
    }

    public static final String DEFAULT_CONFIG_FILE =
            "config/qa.properties";

    public static final String SCREENSHOT_DIRECTORY =
            "target/screenshots/";

    public static final String ALLURE_RESULTS_DIRECTORY =
            "target/allure-results/";

    public static final int EXPLICIT_WAIT_SECONDS = 25;

    public static final int PAGE_LOAD_TIMEOUT_SECONDS = 75;

    public static final int SCRIPT_TIMEOUT_SECONDS = 30;

    public static final int API_TIMEOUT_SECONDS = 40;

    public static final int RETRY_COUNT = 2;

    public static final int API_RESPONSE_TIME_LIMIT = 3000;

    public static final String BROWSER_CHROME = "chrome";

    public static final String BROWSER_FIREFOX = "firefox";

    public static final String LOCAL_EXECUTION = "local";
```

```
    public static final String GRID_EXECUTION = "grid";

    public static final String CI_EXECUTION = "ci";

    public static final String SENSITIVE_FIELDS =
            "password,otp,pin,token";
}
```

One important architectural point:

Fixed framework constants should contain actual framework constants, not environment-specific values.

URLs, Grid URLs, environment-specific paths, etc. remain in properties files.

6. Configuration

Example `qa.properties`:

```
# Application
base.url=https://qa.example.com

# Browser
browser=chrome
browser.version=stable

# Execution
execution.mode=local
headless=false

# Window
window.width=1920
window.height=1080
maximize=false

# Grid
```

```
grid.enabled=false
grid.url=http://localhost:4444

# Test data
excel.path=src/test/resources/testdata/excel/login.xlsx
json.path=src/test/resources/testdata/json/login.json

# Splunk
splunk.url=https://splunk.example.com
splunk.username.secret=splunk-username
splunk.password.secret=splunk-password

# Retry
retry.count=2
```

7. DriverManager

This is one of the most important pieces because you selected **parallel execution by methods**.

Therefore:

```
package com.automation.framework.driver;

import org.openqa.selenium.WebDriver;

public final class DriverManager {

    private DriverManager() {
    }

    /*
     * Each TestNG worker thread gets its own WebDriver.
     */
    private static final ThreadLocal<WebDriver> DRIVER =
            new ThreadLocal<>();
```

```
    public static void setDriver(WebDriver driver) {
        DRIVER.set(driver);
    }

    public static WebDriver getDriver() {
        WebDriver driver = DRIVER.get();

        if (driver == null) {
            throw new IllegalStateException(
                    "WebDriver is not initialized for the current
thread.");
        }

        return driver;
    }

    public static void quitDriver() {

        WebDriver driver = DRIVER.get();

        if (driver != null) {
            driver.quit();
            DRIVER.remove();
        }
    }

    public static boolean isDriverAvailable() {
        return DRIVER.get() != null;
    }
}
```

Why `ThreadLocal`?

Suppose TestNG starts:

```
Thread-1 → LoginTest
Thread-2 → SearchTest
Thread-3 → PaymentTest
```

Each must have:

```
Thread-1 → Driver-1
Thread-2 → Driver-2
Thread-3 → Driver-3
```

Without `ThreadLocal`, tests can accidentally control each other's browsers.

8. Driver Factory

```
package com.automation.framework.driver;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.remote.DesiredCapabilities;
import org.openqa.selenium.remote.RemoteWebDriver;

import java.net.MalformedURLException;
import java.net.URI;

public final class DriverFactory {

    private DriverFactory() {
    }

    public static WebDriver createDriver(
            String browser,
            boolean gridEnabled,
            String gridUrl) {

        if (gridEnabled) {
            return createRemoteDriver(browser, gridUrl);
        }

        return createLocalDriver(browser);
    }
```

```
    private static WebDriver createLocalDriver(String browser) {

        return switch (browser.toLowerCase()) {

            case "chrome" -> new ChromeDriver();

            case "firefox" -> new FirefoxDriver();

            default -> throw new IllegalArgumentException(
                    "Unsupported browser: " + browser);
        };
    }

    private static WebDriver createRemoteDriver(
            String browser,
            String gridUrl) {

        try {

            DesiredCapabilities capabilities =
                    new DesiredCapabilities();

            capabilities.setBrowserName(browser);

            return new RemoteWebDriver(
                    URI.create(gridUrl).toURL(),
                    capabilities);

        } catch (MalformedURLException e) {

            throw new RuntimeException(
                    "Invalid Selenium Grid URL: " + gridUrl, e);
        }
    }
}
```

Why Selenium Manager?

For local execution, modern Selenium can automatically manage browser drivers.

So you don't need:

```
chromedriver.exe
geckodriver.exe
```

checked into your repository.

9. BaseTest

```
package com.automation.tests.base;

import com.automation.framework.driver.DriverFactory;
import com.automation.framework.driver.DriverManager;
import org.openqa.selenium.WebDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

public abstract class BaseTest {

    @BeforeMethod
    public void setUp() {

        WebDriver driver = DriverFactory.createDriver(
                "chrome",
                false,
                null
        );

        DriverManager.setDriver(driver);

        driver.manage().window().maximize();
    }

    @AfterMethod
    public void tearDown() {

        if (DriverManager.isDriverAvailable()) {
```

```
            DriverManager.quitDriver();
        }
    }
}
```

The production implementation will obviously replace the hard-coded values with your configuration layer.

10. BasePage

Your selected architecture needs an important refinement here.

You selected both:

- Page Objects
- parallel method execution
- singleton/stateless behavior
- ThreadLocal driver

The safest implementation is:

Page Objects themselves should not store mutable WebDriver state.

`BasePage` can obtain the current thread's driver from `DriverManager`.

```
package com.automation.framework.pages;

import com.automation.framework.driver.DriverManager;
import com.automation.framework.waits.WaitUtils;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public abstract class BasePage {

    protected final WebDriver driver;

    protected BasePage() {
        this.driver = DriverManager.getDriver();
```

```
    }

    protected void click(By locator) {

        WaitUtils.waitForClickable(locator).click();
    }

    protected void type(By locator, String value) {

        WaitUtils.waitForVisible(locator)
                .clear();

        driver.findElement(locator)
                .sendKeys(value);
    }

    protected String getText(By locator) {

        return WaitUtils.waitForVisible(locator)
                .getText();
    }
}
```

However, because you selected strict per-test browser isolation, I recommend **Page Objects be instantiated per test**, rather than true global singletons.

That avoids a subtle race condition.

11. Wait Utility

```
package com.automation.framework.waits;

import com.automation.framework.driver.DriverManager;
import com.automation.framework.constants.FrameworkConstants;

import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;

public final class WaitUtils {

    private WaitUtils() {
    }

    private static WebDriverWait getWait() {

        return new WebDriverWait(
                DriverManager.getDriver(),
                Duration.ofSeconds(
                        FrameworkConstants.EXPLICIT_WAIT_SECONDS
                )
        );
    }

    public static WebElement waitForVisible(By locator) {

        return getWait().until(
                ExpectedConditions.visibilityOfElementLocated(locator)
        );
    }

    public static WebElement waitForClickable(By locator) {

        return getWait().until(
                ExpectedConditions.elementToBeClickable(locator)
        );
    }

    public static boolean waitForUrl(String expectedUrl) {

        return getWait().until(
                ExpectedConditions.urlToBe(expectedUrl)
```

```
        );
    }
}
```

This follows your requirement:

```
No Thread.sleep()
       ↓
Explicit Wait
       ↓
Reusable Wait Utility
```

12. Page Object

```
package com.automation.tests.pages;

import com.automation.framework.pages.BasePage;
import org.openqa.selenium.By;

public class LoginPage extends BasePage {

    private final By username =
            By.id("username");

    private final By password =
            By.id("password");

    private final By loginButton =
            By.id("login");

    private final By loginPageTitle =
            By.tagName("h1");

    public LoginPage() {
        super();
    }
```

```
    public LoginPage open(String url) {

        driver.get(url);

        return this;
    }

    public HomePage login(
            String user,
            String pwd) {

        type(username, user);
        type(password, pwd);
        click(loginButton);

        return new HomePage();
    }

    public boolean isLoaded() {

        return driver.getTitle()
                .equals("Login");
    }
}
```

This demonstrates your selected:

- By
- private locators
- exact title validation
- Page Object
- navigation returning destination Page Object

13. Component Object

For your selected reusable components:

```
package com.automation.tests.components;

import com.automation.framework.driver.DriverManager;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class HeaderComponent {

    private final WebDriver driver;

    private final By profile =
            By.id("profile");

    public HeaderComponent() {
        this.driver = DriverManager.getDriver();
    }

    public void openProfile() {

        driver.findElement(profile)
                .click();
    }
}
```

14. TestNG DataProvider

Example:

```
@DataProvider(name = "loginData")
public Object[][] loginData() {

    return new Object[][] {
            {"user1", "password1"},
            {"user2", "password2"},
            {"user3", "password3"}
    };
}
```

Then:

```
@Test(
        dataProvider = "loginData",
        description = "Verify successful login"
)
public void verifyLogin(
        String username,
        String password) {

    LoginPage loginPage =
            new LoginPage();

    HomePage homePage =
            loginPage
                    .open(config.getBaseUrl())
                    .login(username, password);

    Assert.assertEquals(
            homePage.getPageTitle(),
            "Home"
    );
}
```

15. Excel Data Architecture

Your selected design:

```
login.xlsx
│
├── Login
│
├── Registration
│
└── Search
```

Each module has its own worksheet.

For environment separation:

```
Login
---------------------------------
QA dataset
...
---------------------------------
UAT dataset
...
```

You selected **separate environment sheets**, so the actual implementation can use:

```
Login_QA
Login_UAT
Registration_QA
Registration_UAT
```

The DataProvider then loads the selected environment's sheet.

16. JSON Architecture

Example:

```
{
  "qa": [
    {
      "username": "user1",
      "password": "password1"
    },
    {
      "username": "user2",
      "password": "password2"
    }
  ],
  "uat": [
    {
      "username": "uatuser1",
      "password": "password1"
```

```
    }
  ]
}
```

Jackson can map this to Java POJOs.

```
public class LoginData {

    private String username;
    private String password;

    public LoginData() {
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

17. RetryAnalyzer

You selected **2 retries**.

```
package com.automation.framework.retry;

import org.testng.IRetryAnalyzer;
```

```
import org.testng.ITestResult;

public class RetryAnalyzer
        implements IRetryAnalyzer {

    private int retryCount = 0;

    private static final int MAX_RETRY = 2;

    @Override
    public boolean retry(ITestResult result) {

        if (retryCount < MAX_RETRY) {

            retryCount++;

            return true;
        }

        return false;
    }
}
```

Usage:

```
@Test(
    retryAnalyzer = RetryAnalyzer.class
)
public void verifyLogin() {
    // test
}
```

Execution:

```
Attempt 1 → Failed
Attempt 2 → Failed
Attempt 3 → Failed
       ↓
Final Failure
```

```
       ↓
testng-failed.xml available
```

18. Failure Screenshot

Your selected behavior:

Screenshot only on failure.

```
public static void captureScreenshot(
        String testName) {

    WebDriver driver =
            DriverManager.getDriver();

    File source =
            ((TakesScreenshot) driver)
                    .getScreenshotAs(
                            OutputType.FILE);

    Path destination =
            Paths.get(
                    "target/screenshots",
                    testName + ".png"
            );

    try {

        Files.createDirectories(
                destination.getParent()
        );

        Files.copy(
                source.toPath(),
                destination,
                StandardCopyOption.REPLACE_EXISTING
        );
```

```
    } catch (IOException e) {

        throw new FrameworkException(
                "Unable to save screenshot",
                e
        );
    }
}
```

The TestNG listener then attaches this screenshot to Allure.

19. TestNG Listener

```
public class TestListener
        implements ITestListener {

    @Override
    public void onTestFailure(
            ITestResult result) {

        String testName =
                result.getMethod()
                        .getMethodName();

        ScreenshotUtils
                .captureScreenshot(testName);
    }
}
```

Register:

```
<listeners>
    <listener
        class-name=
        "com.automation.framework.listeners.TestListener"/>
</listeners>
```

20. TestNG Parallel Execution

You selected:

Parallel = methods

Example:

```
<suite
        name="UI Regression"
        parallel="methods"
        thread-count="4">

    <test name="UI Tests">

        <classes>

            <class name=
                "com.automation.tests.ui.LoginTest"/>

            <class name=
                "com.automation.tests.ui.SearchTest"/>

        </classes>

    </test>

</suite>
```

The critical design is:

```
TestNG Thread
      ↓
ThreadLocal<WebDriver>
      ↓
Independent Browser
```

21. Maven Group Execution

You selected Surefire group execution.

Examples:

```
mvn clean test -Dgroups=smoke
mvn clean test -Dgroups=regression
```

Multiple groups:

```
mvn clean test -Dgroups=smoke,regression
```

Flaky tests:

```
mvn clean test -Dgroups=flaky
```

And because you selected flaky tests to be excluded from normal regression, they can remain a separate execution group.

22. Selenium Grid

Local:

```
DriverFactory
      ↓
ChromeDriver / FirefoxDriver
```

Grid:

```
DriverFactory
      ↓
RemoteWebDriver
      ↓
http://grid:4444
      ↓
Selenium Grid
```

Configuration:

```
grid.enabled=true
grid.url=http://localhost:4444
```

23. Allure Architecture

Your selected evidence model is:

```
TestNG
  │
  ├── status
  ├── exception
  └── failure
       │
       ▼
TestListener
       │
       ├── Screenshot
       └── Allure attachment
```

You also selected:

- business-level Allure steps
- dynamic environment metadata
- failure evidence
- final result as primary retry result

So avoid putting:

```
@Step
public void click() {}
```

on every low-level Selenium operation.

Instead:

```
@Step("Login to application")
public HomePage login(...) {
```

```
    ...
}
```

This keeps Allure readable.

24. Important Architecture Decision

One thing I would **not** implement literally from the questionnaire is a globally shared singleton Page Object with a mutable driver.

For example, this is unsafe:

```
static LoginPage loginPage;
```

shared by:

```
Thread 1
Thread 2
Thread 3
```

because all three can potentially modify the same object state.

Your stronger requirements are:

```
Parallel methods
+
One browser per test
+
ThreadLocal driver
+
Maintainability
```

Therefore the recommended implementation is:

```
Test
 ↓
new LoginPage()
 ↓
DriverManager.getDriver()
```

```
 ↓
current ThreadLocal WebDriver
```

This preserves the **simplicity** you selected without compromising parallel execution.

25. Implementation Roadmap

We'll implement the framework in this order:

```
STEP 1
Maven + Java 21
        ↓
STEP 2
Configuration Manager
        ↓
STEP 3
DriverManager + ThreadLocal
        ↓
STEP 4
DriverFactory
        ↓
STEP 5
BaseTest
        ↓
STEP 6
BasePage + WaitUtils
        ↓
STEP 7
Page Objects
        ↓
STEP 8
Reusable Components
        ↓
STEP 9
Excel DataProvider
        ↓
STEP 10
```

```
JSON DataProvider
        ↓
STEP 11
POJO mapping
        ↓
STEP 12
TestNG listeners
        ↓
STEP 13
Allure
        ↓
STEP 14
RetryAnalyzer
        ↓
STEP 15
Selenium Grid
        ↓
STEP 16
WebDriverListener
        ↓
STEP 17
AWS Secrets Manager
        ↓
STEP 18
Splunk OTP
        ↓
STEP 19
Rest Assured module
        ↓
STEP 20
Final CI/CD-ready execution
```

This gives us a genuinely scalable framework rather than just a collection of Selenium utilities.

The next implementation step should therefore be **Step 1: create the Maven project and complete `pom.xml`, followed by the configuration and driver layers**, before moving into Page Objects and data-driven testing.